

Python Basics Reference

Elementary Scripts, Syntax, and Use Cases

Covers: variables, None/null checks, collections,
conditionals, loops, functions, files,
pandas & numpy essentials for ML labs.

1. Variables and Data Types

Python is dynamically typed. You do not declare types explicitly.

```
name = 'Ojas'           # str
age = 20                # int
height = 5.9            # float
is_student = True       # bool
nothing = None          # NoneType (Python's null)
```

Use case: Store different kinds of values in variables.

Check type

```
type(age)               # <class 'int'>
isinstance(age, int)    # True
```

Use case: Verify what kind of data you are working with.

Convert types

```
int('42')              # 42
float('3.14')           # 3.14
str(100)                 # '100'
bool(0)                  # False
bool(1)                  # True
list('abc')              # ['a', 'b', 'c']
```

Use case: Convert user input or file data to the type you need.

2. None (Null) Checks

Python uses None, not null. Always compare with 'is' or 'is not'.

```
x = None

if x is None:
    print('x is null')

if x is not None:
    print('x has a value')
```

Use case: Check if a variable has no value yet.

Note: Use 'is None', NOT 'x == None'. 'is' checks identity correctly.

Default values for None

```
def greet(name=None):
    if name is None:
        name = 'Guest'
    return f'Hello, {name}'

value = some_var if some_var is not None else 0
```

Use case: Provide fallback when input is missing.

Safe access

```
data = {'score': None}
score = data.get('score')      # None if missing
score = data.get('score', 0)   # 0 if missing

name = user.get('name') or 'Unknown'
```

Use case: Avoid KeyError and handle missing dictionary keys.

3. Strings

```
s = 'hello'
s.upper()      # 'HELLO'
s.lower()      # 'hello'
s.strip()      # remove spaces
s.replace('l', 'L') # 'heLlo'
s.split(',')   # split by comma
','.join(['a','b']) # 'a,b'
'age' in s     # True/False
len(s)         # 5
```

Use case: Clean and format text data.

f-strings (formatted strings)

```
name = 'Ojas'
score = 95.5
f'Name: {name}, Score: {score:.1f}%'
f'Price: ${1000:,.2f}'    # $1,000.00
```

Use case: Print readable output with variables embedded.

4. Lists

```
nums = [1, 2, 3, 4, 5]
nums[0]      # first item: 1
nums[-1]     # last item: 5
nums[1:3]    # slice: [2, 3]
nums.append(6) # add to end
nums.insert(0, 0)
nums.remove(3) # remove value
nums.pop()    # remove last
len(nums)
3 in nums     # membership check
```

Use case: Store ordered collections that can change.

List comprehension

```
squares = [x**2 for x in range(1, 6)]
evens = [x for x in nums if x % 2 == 0]
pairs = [a+b for a in 'abc' for b in 'def']
```

Use case: Create new lists in one line (Assignment 1 Q4).

5. Dictionaries and Sets

```
student = {'name': 'Ojas', 'age': 20}
student['name']      # get value
student.get('grade', 'N/A')
student.keys()
student.values()
student.items()
student['grade'] = 9
'name' in student    # key exists?

a = {1, 2, 3}
b = {3, 4, 5}
a & b    # intersection: {3}
a | b    # union: {1,2,3,4,5}
a - b    # difference: {1, 2}
```

Use case: Dicts for key-value data (Netflix views). Sets for unique items and set math (Q1 sports).

Dictionary comprehension

```
{x: x**3 for x in range(1, 101) if x % 3 == 0}
```

Use case: Build dictionaries quickly (Assignment 1 Q3).

6. Conditionals (if / elif / else)

```
if score >= 90:
    grade = 'A'
elif score >= 75:
    grade = 'B'
else:
```

```
grade = 'C'
```

```
# one-liner
status = 'pass' if score >= 40 else 'fail'
```

Use case: Make decisions based on conditions.

Truthy and Falsy values

Falsy: None, 0, 0.0, "", [], {}, set(), False

Everything else is truthy.

```
if my_list:
    print('list is not empty')

if not name:
    print('name is empty or None')
```

Use case: Quick checks without writing 'is not None' every time.

7. Loops

```
for i in range(5):          # 0,1,2,3,4
    print(i)

for item in my_list:
    print(item)

for key, val in student.items():
    print(key, val)

count = 0
while count < 5:
    count += 1

for i in range(10):
    if i == 3:
        continue    # skip
    if i == 7:
        break        # stop loop
```

Use case: Repeat actions over data.

enumerate and zip

```
for i, val in enumerate(['a', 'b', 'c']):
    print(i, val)    # 0 a, 1 b, 2 c

for a, b in zip([1,2], ['x','y']):
    print(a, b)      # 1 x, 2 y
```

Use case: Loop with index, or combine two lists together.

8. Functions

```
def add(a, b):
    return a + b

def greet(name='Guest'):
    return f'Hello, {name}'

def stats(nums):
    return min(nums), max(nums), sum(nums)/len(nums)

lo, hi, avg = stats([1, 2, 3, 4, 5])
```

Use case: Reusable blocks of code. Default args handle missing inputs.

Lambda (small one-line functions)

```
square = lambda x: x**2
sorted(words, key=lambda x: len(x))
df['LotFrontage'].fillna(df.groupby('Neighborhood')['LotFrontage'].transform(lambda x: x.median()))
```

Use case: Quick functions for sorting or pandas group operations.

9. File Handling

```
# Read text file
with open('file.txt', 'r') as f:
    content = f.read()

# Write text file
with open('out.txt', 'w') as f:
    f.write('Hello')

# Read CSV with pandas
import pandas as pd
df = pd.read_csv('data/train.csv')
df.to_csv('output.csv', index=False)
```

Use case: Load datasets and save results. 'with' auto-closes the file.

10. Error Handling

```
try:
    value = int(user_input)
except ValueError:
    print('Not a valid number')
except KeyError:
    print('Key not found')
else:
    print('Success:', value)
finally:
    print('Always runs')
```

Use case: Handle bad input or missing data without crashing.

11. Pandas: Null / Missing Values

In pandas, missing values are NaN (Not a Number), not None.

```
import pandas as pd
import numpy as np

df.isnull()          # True where missing
df.notnull()         # True where present
df.isna()            # same as isnull()
df.notna()           # same as notnull()

df.isnull().sum()     # count missing per column
df.isnull().sum().sum() # total missing

df['col'].isnull().any() # any missing in column?
df['col'].notna().all()  # all values present?
```

Use case: Find missing data before modeling (Assignment 1 Part D).

Fill missing values

```
df['num_col'].fillna(df['num_col'].median())
df['cat_col'].fillna(df['cat_col'].mode()[0])
df['cat_col'].fillna('Missing')

df.dropna()          # drop rows with any NaN
df.dropna(subset=['SalePrice']) # drop if target missing

# LotFrontage by neighborhood
df['LotFrontage'] = df.groupby('Neighborhood')['LotFrontage'].transform(
    lambda x: x.fillna(x.median())
)
```

Use case: Impute missing values (median for numeric, mode for categorical).

Filter rows with/without nulls

```
df[df['SalePrice'].notna()]      # rows where price exists
df[df['PoolQC'].isnull()]       # rows where PoolQC is missing
df.drop(columns=['Id'])         # remove a column
```

Use case: Clean dataset by keeping only valid rows.

12. Pandas Essentials

```
df.head(10)          # first 10 rows
df.tail()            # last 5 rows
df.shape             # (rows, cols)
df.columns           # column names
df.dtypes            # data types
df.info()            # summary
df.describe()        # numeric stats

df['SalePrice']       # one column
df[['A', 'B']]        # multiple columns
df.loc[0]             # row by label
df.iloc[0]           # row by index
df[df['Age'] > 18]     # filter rows
```

Use case: Explore and slice tabular data (Assignment 1 Part A).

Useful operations

```
df['SalePrice'].mean()
df['SalePrice'].median()
df['SalePrice'].std()
df['SalePrice'].min()
df['SalePrice'].max()
df['Neighborhood'].value_counts()
df.select_dtypes(include=['number']).columns
df.select_dtypes(include=['object']).columns
df.corr()             # correlation matrix
```

Use case: Statistics and EDA for ML assignments.

13. NumPy Essentials

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5])
arr.shape
arr.mean()
arr.std()
arr.min(), arr.max()

np.zeros((3, 3))      # 3x3 zeros
np.ones((2, 4))       # 2x4 ones
np.arange(0, 10, 2)    # [0,2,4,6,8]
np.linspace(0, 1, 5)   # 5 evenly spaced

arr[arr > 3]           # filter
np.where(arr > 3)       # indices where True
np.isin(arr, [1, 5])   # element in list?
np.unique(arr)         # unique values
np.sort(arr)           # sorted copy
```

Use case: Numerical arrays for ML lab NumPy questions (Assignment 1 Q7-Q19).

Common NumPy functions (from your assignments)

```
np.percentile(arr, 50)          # median
np.argsort(arr)                 # sort indices
np.partition(arr, 4)            # partition at position
np.linalg.norm(a - b)          # euclidean distance
np.diff(arr, prepend=[0], append=[200])
np.abs(np.round(arr))          # round + absolute
arr[~np.isnan(arr)]            # remove NaN
```

Use case: Directly maps to Assignment 1 NumPy exercises.

14. Common Imports for ML Labs

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_squared_error
from sklearn.tree import DecisionTreeClassifier
from scipy import stats
```

Use case: Standard imports used across all 3 assignments.

15. Quick Syntax Cheat Sheet

Task	Syntax	Use case
Check if None	<code>x is None</code>	Variable has no value
Check if not None	<code>x is not None</code>	Variable has a value
Check if empty list	<code>not my_list</code>	List is empty
Check membership	<code>x in my_list</code>	x exists in list
Safe dict get	<code>d.get('key', default)</code>	Avoid KeyError
Ternary if	<code>a if cond else b</code>	One-line if/else
Pandas missing?	<code>df.isnull()</code>	Find NaN cells
Pandas has value?	<code>df.notna()</code>	Find non-missing
Count missing	<code>df.isnull().sum()</code>	Per column count
Fill missing	<code>df.fillna(value)</code>	Replace NaN
Filter rows	<code>df[df['col'] > 5]</code>	Conditional rows
Group stats	<code>df.groupby('col').mean()</code>	Stats per group
Train/test split	<code>train_test_split(X, y, test_size=0.2)</code>	ML evaluation
Read CSV	<code>pd.read_csv('file.csv')</code>	Load dataset
Save CSV	<code>df.to_csv('out.csv', index=False)</code>	Export results

Common mistakes to avoid

1. Using `== None` instead of `is None`
2. Forgetting to call a function: `len` vs `len()`
3. Modifying a list while looping over it
4. Not handling missing values before sklearn models
5. Using `df['col'] = ...` without `copy` -> `SettingWithCopyWarning`
6. Comparing strings with `==` when you need `.strip()` first
7. Dividing by zero -> check denominator first
8. Not setting `random_state` in `train_test_split` (results change each run)